

Day 23/8/2026 Technical Report

1) Software

1- Stereo Vision Subsystem

- **Goal:** Derive 3D coordinates and measure real-world distances.
- **Calibration & Matching:** Extracted intrinsic matrix, baseline B , and offset D_{off} . Used SIFT and BFMatcher with Lowe's Ratio Test (< 0.75) to filter reliable key points
- **Rectification & Depth:** Computed Fundamental/Essential matrices to derive camera pose (R, T). Applied (stereoRectify) and SGBM scanline matching to construct disparity maps.
- **3D Output:** Calculated depth using ($Z = \frac{F*B}{D + D_{off}}$) to project image points into 3D space (X, Y, Z) for distance measurement.

2- Object Detection Subsystem

- **Goal:** Detect target objects (water bottles) in real time for navigation.
- **Dataset & Training:** Annotated custom data via Roboflow. Trained a lightweight **YOLOv26n (Nano)** model in Colab using manual tuning ($l_{r0}=0.01$, batch =16, img =640, patience \$=20\$) and multi-scenario data augmentations.
- **Deployment:** Verified via confusion matrices (mode=val). Integrated OpenCV with a wireless phone camera feed (Camo), running inference at 1280 with a 0.2 confidence threshold to draw real-time bounding boxes.

3- Next Steps

- Fuse object bounding boxes with stereo depth maps to calculate exact 3D spatial coordinates of detected targets.
- Work on React : Frontend, backend and GUI

2) Firmware

1- Station Communication:

- **Goal:** Station Communication (wireless link between car MCU and laptop GUI)
- **Chip:** ATmega8A (8KB flash, 1KB SRAM, single hardware UART)
- **Module comparison:** Evaluated HC-05 (Bluetooth) vs ESP-01 (WiFi) for the wireless link.
- **Decision:** HC-05 (Bluetooth)
- **Reasoning:**

Communicates over plain UART no AT-command parsing needed on the MCU, unlike ESP-01 which requires building/parsing text commands (AT+CIPSTART, etc.) for every connection and data transfer.

Minimal flash/RAM overhead, leaving more headroom for motor control and sensor logic on the ATmega8A's limited memory.

Simpler wiring only needs a voltage divider on RX for 3.3V logic level; ESP-01 also needs this plus a more stable power supply for its higher current draw.

Pairs and appears as a standard serial (COM) port on the laptop, so the Python backend can read it directly via pyserial same as a wired connection. ESP-01 would require socket-based (TCP) communication instead, adding complexity to the backend.

Sufficient range and bandwidth for the task (small motion commands + sensor readings); the live camera feed runs separately via Python, not through the MCU, so WiFi's extra bandwidth isn't needed here.

- **Next steps:**

Wire HC-05 to ATmega8A UART, configure via AT mode (name/PIN/ baud rate), define the packet protocol for commands and sensor data, and confirm baud rate choice (9600 or 38400) based on clock error at 8MHz.

2- Sensor Data acquisition:

I completed setting up the coding environment , researched adc and i2c to have better understanding of how the MCU will acquire data from sensors, and created the voltage data acquisition code structure

3) Hardware

-----Designed the Schematic for the MCU Board-----

1- Power section

Used a normal terminal block for battery connection , an LM7805 with decoupling capacitors for power input for the ICs and Modules (Big Bulk cap on input 100uF and 10uF on output) , used an LED for indication that input power is on

2- MCU section

Used The ATMEGA8A chip which is found on makers website <https://makerselectronics.com/product/atmega8a-pu-8-bit-avr-microcontroller-dip-28-original/> which is the original from microchip , used microchip's footprint and schematic with a slight modification to make it adjustable on the footprint for the 28 pin socket

Used decoupling cap (100nF) as close as possible physically to the MCU, used a ferrite bead + capacitor filter on the AVCC pin to get a clear non glitching ADC (10uH + 100nF which is used commercially) , made the reset circuit with the debouncing cap (100nF) , labeled each pin as found on makers website

3- IMU section

Used the module which is found on makers website <https://makerselectronics.com/product/mpu-6050-imu-sensor-module-with-6-dof-gyroscope-accelerometer/> , Made the 3D body of 2 parts from Snapeda and 3DCentral

Made sure of the footprint to match the one in makers

Connected SDA and SCL for I2C , used the default register 0x68 , and made sure of it by wiring AD0 to GND